Customer Churn Prediction

In [ ]:
```
!pip install catboost
```

In [15]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings
warnings.filterwarnings('ignore')

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler


from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier


from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier
from catboost import CatBoostClassifier
from sklearn import metrics
from sklearn.metrics import roc_curve
from sklearn.metrics import recall_score, confusion_matrix, precision_score, f1_

from sklearn.ensemble import VotingClassifier
```

In [16]:
```
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import f1_score, precision_score, recall_score, fbeta_score
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import ShuffleSplit
from sklearn.model_selection import KFold
from sklearn import feature_selection
from sklearn import model_selection
from sklearn import metrics
from sklearn.metrics import classification_report, precision_recall_curve
```

```
from sklearn.metrics import auc, roc_auc_score, roc_curve
from sklearn.metrics import make_scorer, recall_score, log_loss
from sklearn.metrics import average_precision_score
#Standard libraries for data visualization:
```

In [17]:
```
data = pd.read_csv("data.csv")
data.head()
```

Out[17]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Mul
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	

5 rows × 21 columns

In [18]:
```
data.isnull().any().any()
```

Out[18]:
```
np.False_
```

In [19]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

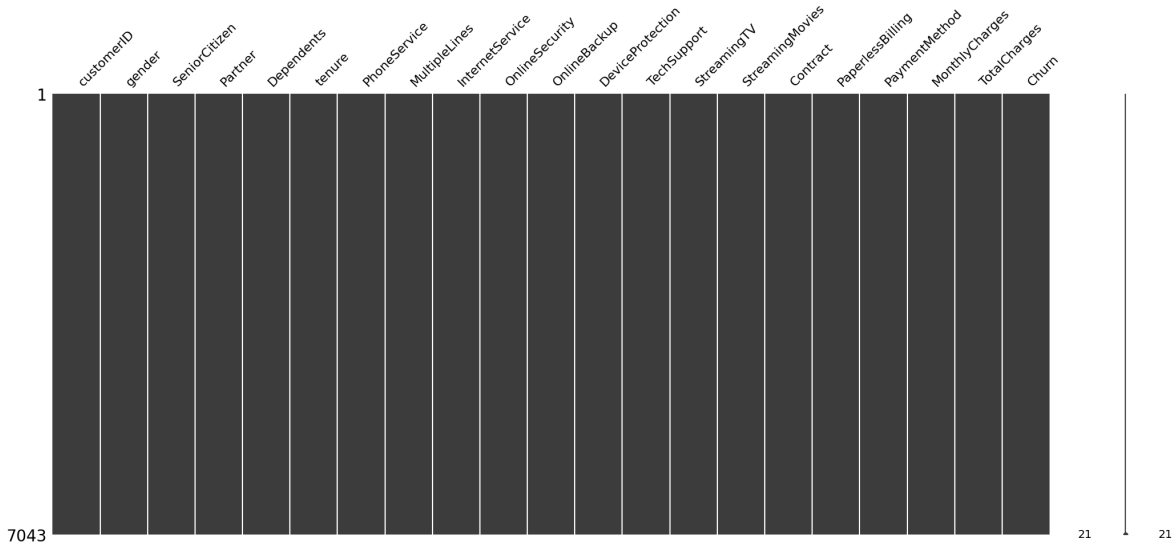
In [20]:
```
data.shape
```

Out[20]: (7043, 21)

Visualize missing values

In [21]:
```
import missingno as msno
msno.matrix(data)
```

Out[21]: <Axes: >



In [22]:
```
data = data.drop(["customerID"], axis = 1)
data.head()
```

Out[22]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	In
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

In [23]:

```
data[data["TotalCharges"] == ' ']
```

Out[23]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
488	Female	0	Yes	Yes	0	No	No phone service
753	Male	0	No	Yes	0	Yes	No
936	Female	0	Yes	Yes	0	Yes	No
1082	Male	0	Yes	Yes	0	Yes	Yes
1340	Female	0	Yes	Yes	0	No	No phone service
3331	Male	0	Yes	Yes	0	Yes	No
3826	Male	0	Yes	Yes	0	Yes	Yes
4380	Female	0	Yes	Yes	0	Yes	No
5218	Male	0	Yes	Yes	0	Yes	No
6670	Female	0	Yes	Yes	0	Yes	Yes
6754	Male	0	No	Yes	0	Yes	Yes

In [24]:

```
data['TotalCharges'] = pd.to_numeric(data.TotalCharges, errors='coerce')
data.isnull().sum()
```

Out[24]:

	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	11
Churn	0

dtype: int64

There are 11 records with missing Total charges

In [25]:
```
data[data["tenure"] == 0]
```

Out[25]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
488	Female	0	Yes	Yes	0	No	No phone service
753	Male	0	No	Yes	0	Yes	No
936	Female	0	Yes	Yes	0	Yes	No
1082	Male	0	Yes	Yes	0	Yes	Yes
1340	Female	0	Yes	Yes	0	No	No phone service
3331	Male	0	Yes	Yes	0	Yes	No
3826	Male	0	Yes	Yes	0	Yes	Yes
4380	Female	0	Yes	Yes	0	Yes	No
5218	Male	0	Yes	Yes	0	Yes	No
6670	Female	0	Yes	Yes	0	Yes	Yes
6754	Male	0	No	Yes	0	Yes	Yes

In [26]:
```
data.drop(labels=data[data["tenure"] == 0].index, axis = 0, inplace = True)
```

In [27]:
```
data.fillna(data["TotalCharges"].mean())
```

Out[27]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
0	Female	0	Yes	No	1	No	No phone service
1	Male	0	No	No	34	Yes	No
2	Male	0	No	No	2	Yes	No
3	Male	0	No	No	45	No	No phone service
4	Female	0	No	No	2	Yes	No
...	...	...	...	...	...	...	...
7038	Male	0	Yes	Yes	24	Yes	Yes
7039	Female	0	Yes	Yes	72	Yes	Yes
7040	Female	0	Yes	Yes	11	No	No phone service
7041	Male	1	Yes	No	4	Yes	Yes
7042	Male	0	No	No	66	Yes	No

7032 rows × 20 columns

In [28]:

```
data['TotalCharges'] = pd.to_numeric(data.TotalCharges, errors='coerce')
data.isnull().sum()
```

Out[28]:

	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	0
Churn	0

dtype: int64

In [29]:
```
data.SeniorCitizen.unique()
```

Out[29]:
```
array([0, 1])
```

In [30]:
```
data.SeniorCitizen = data.SeniorCitizen.map({0: "No", 1: "Yes"})
data.head()
```

Out[30]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	In
0	Female	No	Yes	No	1	No	No phone service	
1	Male	No	No	No	34	Yes	No	
2	Male	No	No	No	2	Yes	No	
3	Male	No	No	No	45	No	No phone service	
4	Female	No	No	No	2	Yes	No	

In [31]:

```
data.InternetService.describe(include=["object", "bool"])
```

Out[31]:

	InternetService
count	7032
unique	3
top	Fiber optic
freq	3096

dtype: object

EDA

In [32]:

```
type_ = ["No", "yes"]
fig = make_subplots(rows=1, cols=1)

fig.add_trace(go.Pie(labels=type_, values=data['Churn'].value_counts(), name="Ch

# Use `hole` to create a donut-like pie chart
fig.update_traces(hole=.4, hoverinfo="label+percent+name", textfont_size=16)

fig.update_layout(
    title_text="Churn Distributions",
    # Add annotations in the center of the donut pies.
    annotations=[dict(text='Churn', x=0.5, y=0.5, font_size=20, showarrow=False)
fig.show()
```

In [33]:
```
data.Churn[data.Churn == "No"].groupby(by = data.gender).count()
```

Out[33]:

gender	Churn
Female	2544
Male	2619

dtype: int64

In [34]:
```
data.Churn[data.Churn == "Yes"].groupby(by = data.gender).count()
```

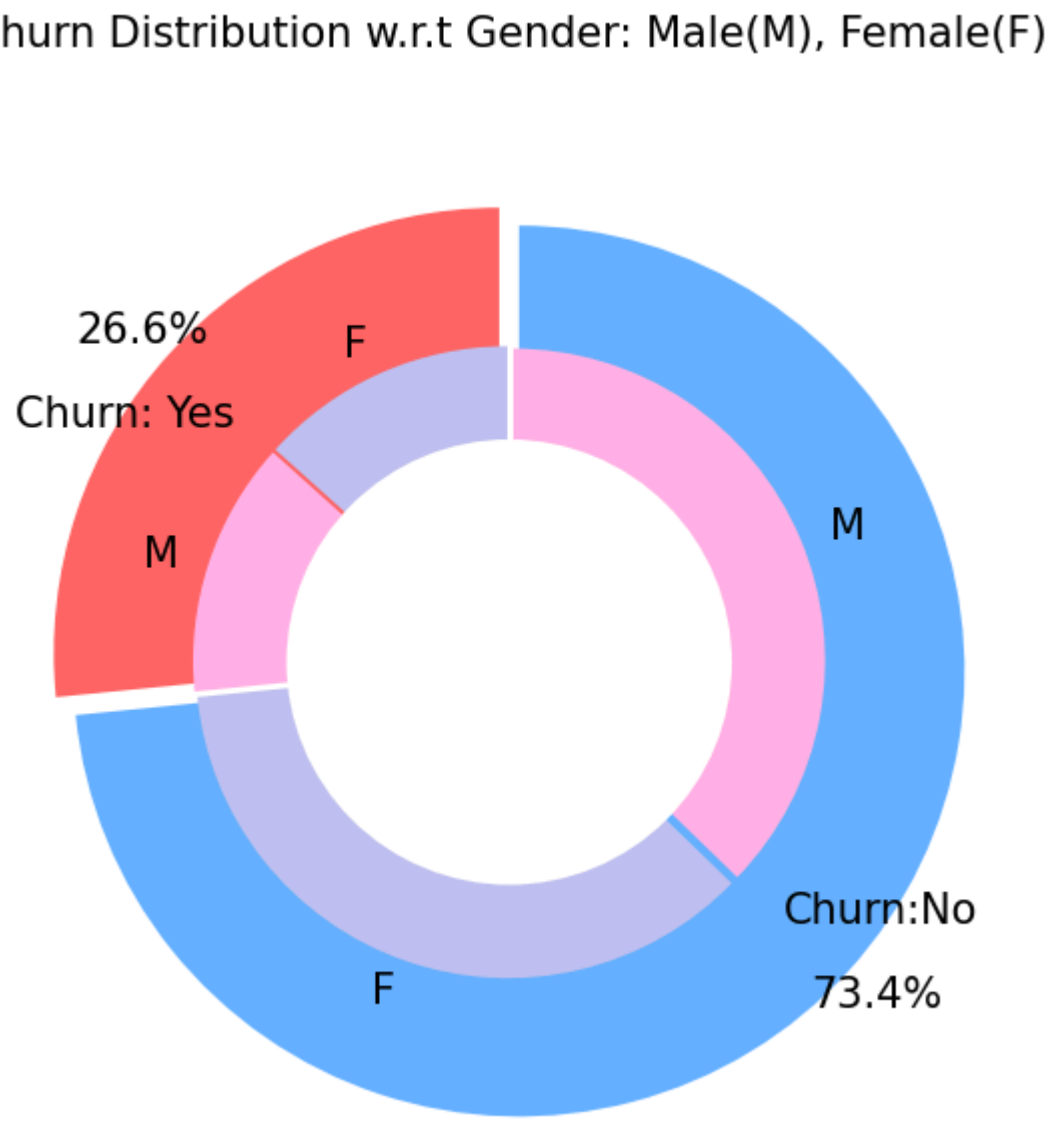
Out[34]:

gender	Churn
Female	939
Male	930

dtype: int64

In [35]:

```
plt.figure(figsize=(6, 6))
labels =["Churn: Yes","Churn:No"]
values = [1869,5163]
labels_gender = ["F","M","F","M"]
sizes_gender = [939,930 , 2544,2619]
colors = ['#ff6666', '#66b3ff']
colors_gender = ['#c2c2f0','#ffb3e6', '#c2c2f0','#ffb3e6']
explode = (0.3,0.3)
explode_gender = (0.1,0.1,0.1,0.1)
textprops = {"fontsize":15}
#Plot
plt.pie(values, labels=labels,autopct='%1.1f%%',pctdistance=1.08, labeldistance=
plt.pie(sizes_gender,labels=labels_gender,colors=colors_gender,startangle=90, ex
#Draw circle
centre_circle = plt.Circle((0,0),5,color='black', fc='white',linewidth=0)
fig = plt.gcf()
fig.gca().add_artist(centre_circle)

plt.title('Churn Distribution w.r.t Gender: Male(M), Female(F)', fontsize=15, y=

# show plot

plt.axis('equal')
plt.tight_layout()
plt.show()
```



In [36]:
```
fig = px.histogram(data, x="Churn", color = "Contract", barmode = "group", title
fig.update_layout(width=700, height=500, bargap=0.2)
fig.show()
```

Customers with monthly contract are more likely to churn

In [37]:
```
labels = data['PaymentMethod'].unique()
values = data['PaymentMethod'].value_counts()

fig = go.Figure(data=[go.Pie(labels=labels, values=values, hole=.3)])
fig.update_layout(title_text="<b>Payment Method Distribution</b>")
fig.show()

fig = px.histogram(data, x="Churn", color="PaymentMethod", title="<b>Customer Pa
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```

In [38]:
```
data[data["gender"]=="Male"][["InternetService", "Churn"]].value_counts()
```

Out[38]:

InternetService	Churn	count
DSL	No	992
Fiber optic	No	910
No	No	717
Fiber optic	Yes	633
DSL	Yes	240
No	Yes	57

dtype: int64

In [39]:
```
data[data["gender"]=="Female"][["InternetService", "Churn"]].value_counts()
```

Out[39]:

InternetService	Churn	count
DSL	No	965
Fiber optic	No	889
No	No	690
Fiber optic	Yes	664
DSL	Yes	219
No	Yes	56

dtype: int64

In [40]:

```
fig = go.Figure()

fig.add_trace(go.Bar(
  x = [['Churn:No', 'Churn:No', 'Churn:Yes', 'Churn:Yes'],
       ["Female", "Male", "Female", "Male"]],
  y = [965, 992, 219, 240],
  name = 'DSL',
))

fig.add_trace(go.Bar(
  x = [['Churn:No', 'Churn:No', 'Churn:Yes', 'Churn:Yes'],
       ["Female", "Male", "Female", "Male"]],
  y = [889, 910, 664, 633],
  name = 'Fiber optic',
))

fig.add_trace(go.Bar(
  x = [['Churn:No', 'Churn:No', 'Churn:Yes', 'Churn:Yes'],
       ["Female", "Male", "Female", "Male"]],
  y = [690, 717, 56, 57],
  name = 'No Internet',
))

fig.update_layout(title_text="<b>Churn Distribution w.r.t. Internet Service and

fig.show()
```

In [41]:

```
color_map = {"Yes": "#FF97FF", "No": "#AB63FA"}
fig = px.histogram(data, x="Churn", color="Dependents", barmode="group", title="
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```

In [42]:

```
color_map = {"Yes": '#FFA15A', "No": '#00CC96'}
fig = px.histogram(data, x="Churn", color="Partner", barmode="group", title="<b>
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```

In [43]:

```
color_map = {"Yes": '#00CC96', "No": '#B6E880'}
fig = px.histogram(data, x="Churn", color="SeniorCitizen", title="<b>Chrun distr
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```

In [44]:

```
color_map = {"Yes": "#FF97FF", "No": "#AB63FA"}
fig = px.histogram(data, x="Churn", color="OnlineSecurity", barmode="group", tit
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```

In [45]:

```
color_map = {"Yes": '#FFA15A', "No": '#00CC96'}
fig = px.histogram(data, x="Churn", color="PaperlessBilling",  title="<b>Chrun d
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```

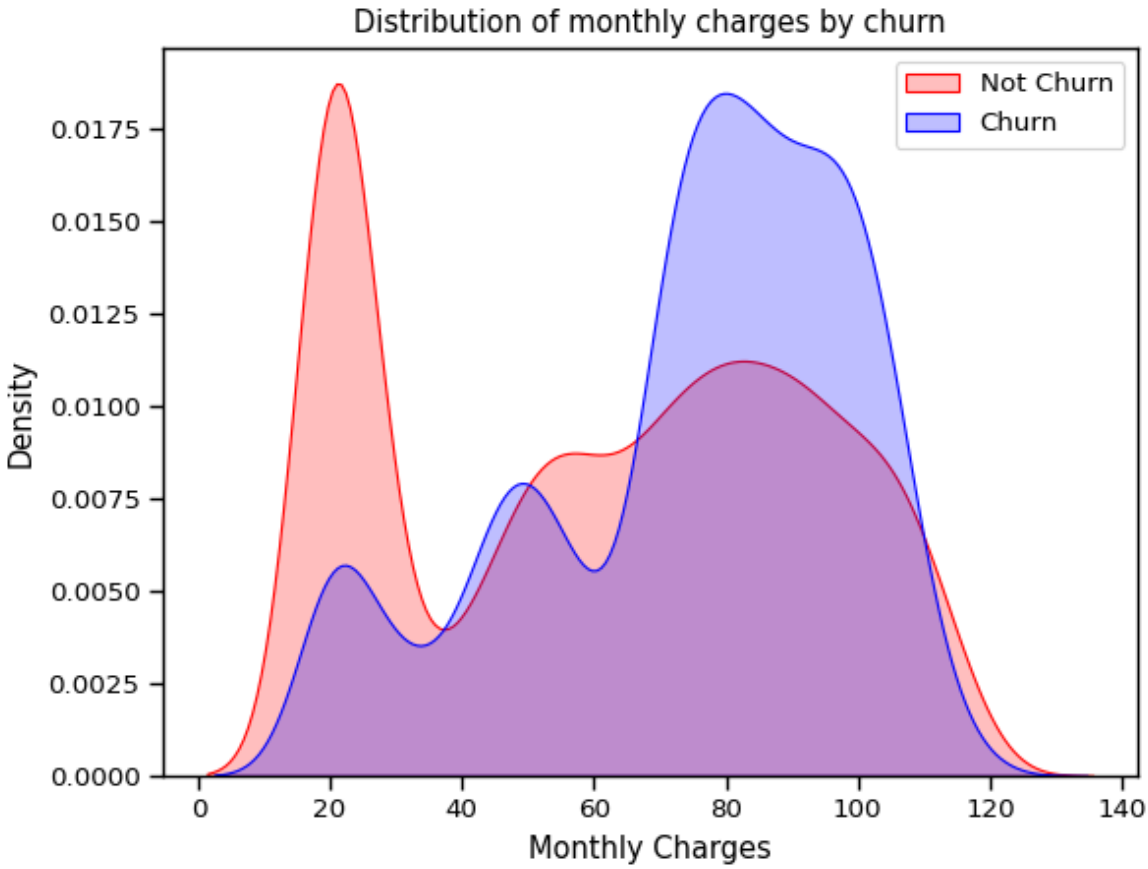
In [46]:
```
fig = px.histogram(data, x="Churn", color="TechSupport",barmode="group",  title=
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```

In [47]:

```
color_map = {"Yes": '#00CC96', "No": '#B6E880'}
fig = px.histogram(data, x="Churn", color="PhoneService", title="<b>Chrun distri
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```

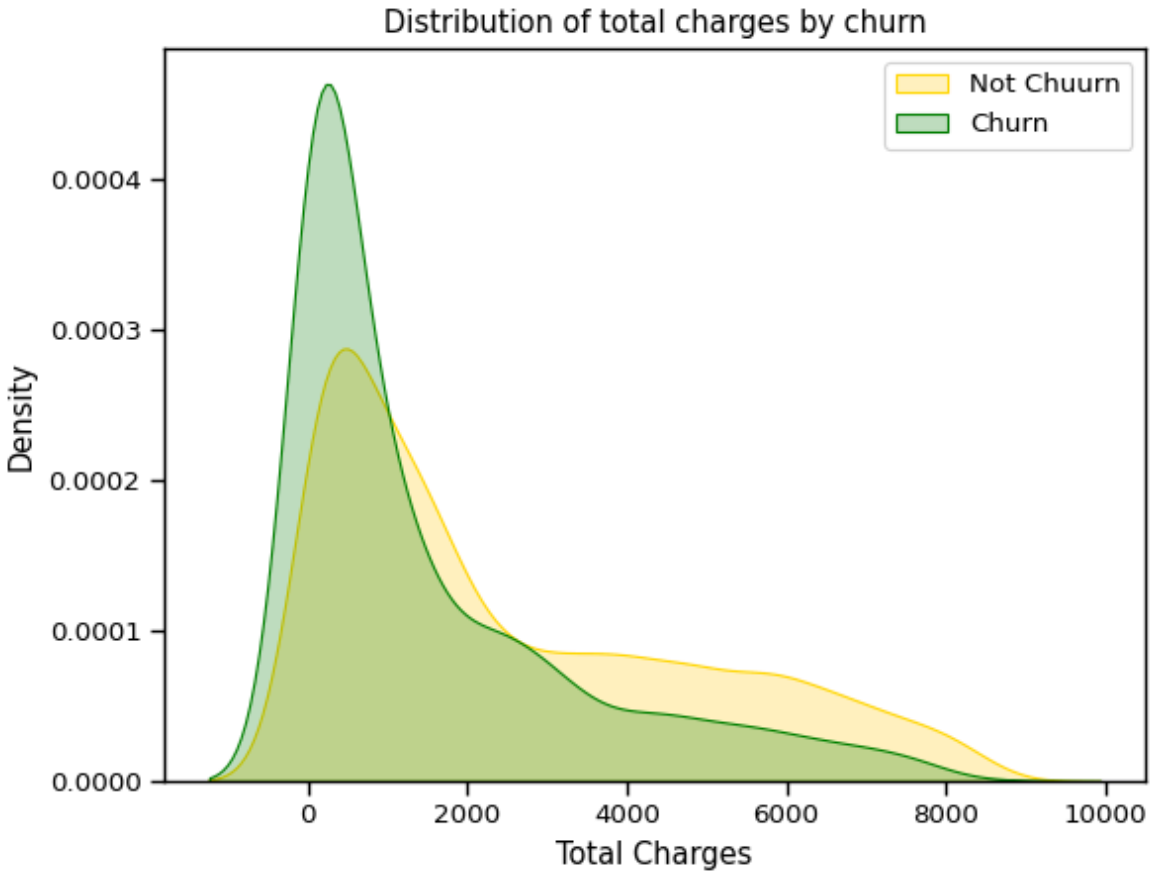
In [48]:

```
sns.set_context("paper",font_scale=1.1)
ax = sns.kdeplot(data.MonthlyCharges[(data["Churn"] == 'No') ],
                 color="Red", shade = True);
ax = sns.kdeplot(data.MonthlyCharges[(data["Churn"] == 'Yes') ],
                 ax =ax, color="Blue", shade= True);
ax.legend(["Not Churn","Churn"],loc='upper right');
ax.set_ylabel('Density');
ax.set_xlabel('Monthly Charges');
ax.set_title('Distribution of monthly charges by churn');
```



Customers with higher monthly charges are more likely to churn

In [49]:
```
ax = sns.kdeplot(data.TotalCharges[(data["Churn"] == 'No') ],
                 color="Gold", shade = True);
ax = sns.kdeplot(data.TotalCharges[(data["Churn"] == 'Yes') ],
                 ax =ax, color="Green", shade= True);
ax.legend(["Not Chuurn","Churn"],loc='upper right');
ax.set_ylabel('Density');
ax.set_xlabel('Total Charges');
ax.set_title('Distribution of total charges by churn');
```



In [50]:
```
fig = px.box(data, x='Churn', y = 'tenure')

# Update yaxis properties
fig.update_yaxes(title_text='Tenure (Months)', row=1, col=1)
# Update xaxis properties
fig.update_xaxes(title_text='Churn', row=1, col=1)

# Update size and title
fig.update_layout(autosize=True, width=750, height=600,
    title_font=dict(size=25, family='Courier'),
    title='<b>Tenure vs Churn</b>',
)

fig.show()
```

New customers are more likely to churn

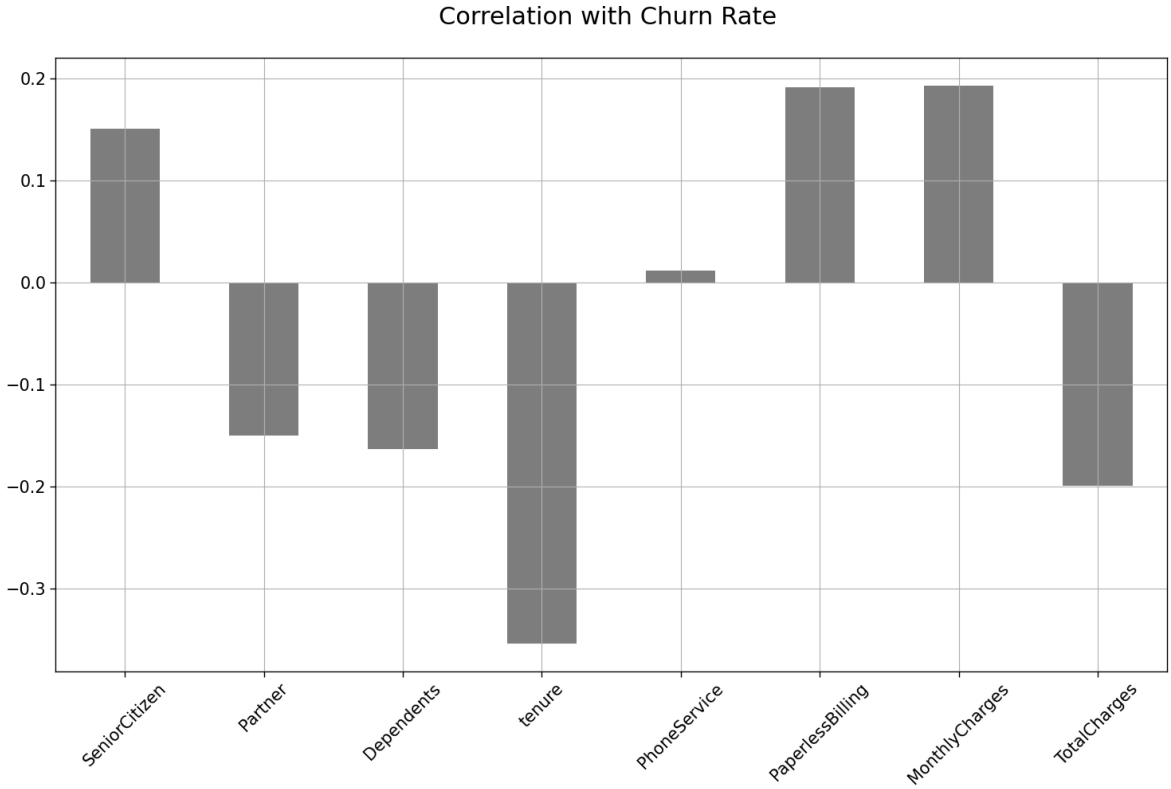
In [51]:
```
#Create a label encoder object
le = LabelEncoder()
# Label Encoding will be used for columns with 2 or less unique
values
le_count = 0
for col in data.columns[1:]:
    if data[col].dtype == 'object':
        if len(list(data[col].unique())) <= 2:
            le.fit(data[col])
            data[col] = le.transform(data[col])
            le_count += 1
print('{} columns were label encoded.'.format(le_count))
```

```
6 columns were label encoded.
```

In [52]:
```
data2 = data[['SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'PaperlessBilling',
        'MonthlyCharges', 'TotalCharges']]

correlations = data2.corrwith(data.Churn)
```

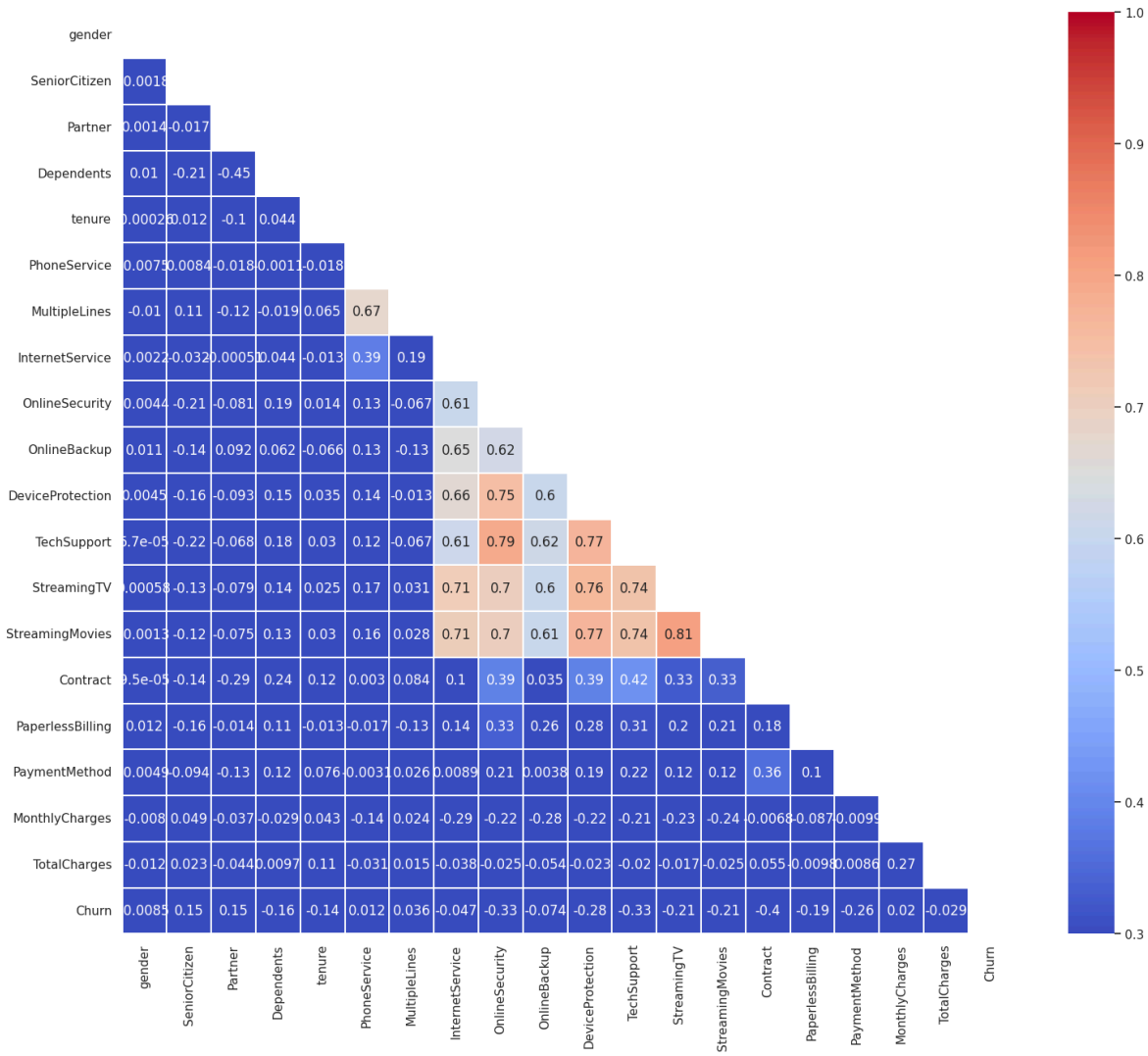
```
correlations = correlations[correlations!=1]
positive_correlations = correlations[correlations >0].sort_values(ascending = Fa
negative_correlations =correlations[correlations<0].sort_values(ascending = Fals

correlations.plot.bar(
        figsize = (18, 10),
        fontsize = 15,
        color = 'grey',
        rot = 45, grid = True)
plt.title('Correlation with Churn Rate \n',
horizontalalignment="center", fontstyle = "normal",
fontsize = "22", fontfamily = "sans-serif")
```

Out[52]: Text(0.5, 1.0, 'Correlation with Churn Rate \n')



In [53]:

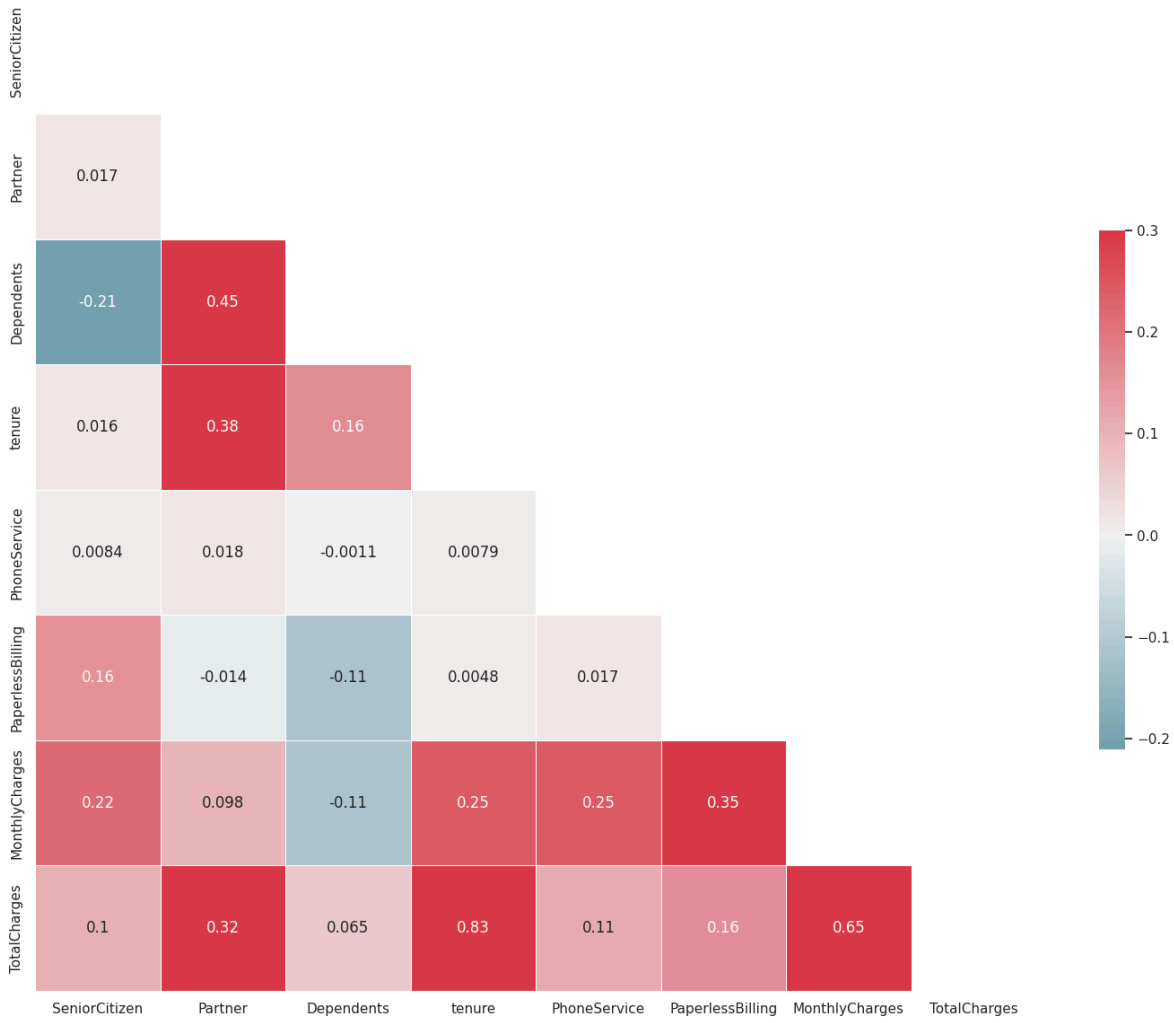
```
#Set and compute the Correlation Matrix:
sns.set(style="white")
plt.figure(figsize=(18, 15))

corr = data.apply(lambda x: pd.factorize(x)[0]).corr()

mask = np.triu(np.ones_like(corr, dtype=bool))

ax = sns.heatmap(corr, mask=mask, xticklabels=corr.columns, yticklabels=corr.col
```



In [54]:

```
#Set and compute the Correlation Matrix:
sns.set(style="white")
corr = data2.corr()

#Generate a mask for the upper triangle:

mask = np.zeros_like(corr, dtype=np.bool)
mask[np.triu_indices_from(mask)] = True

#Set up the matplotlib figure and a diverging colormap:
f, ax = plt.subplots(figsize=(18, 15))
cmap = sns.diverging_palette(220, 10, as_cmap=True)

#Draw the heatmap with the mask and correct aspect ratio:
sns.heatmap(corr, mask=mask, cmap=cmap, vmax=.3, center=0,square=True,annot = Tr
```

Out[54]: <Axes: >



Multicollinearity check using VIF

TODO:

from statsmodels.stats.outliers_influence import variance_inflation_factor data_ = pd.read_csv("data.csv") X = data_[['gender', 'SeniorCitizen', 'Partner', 'Dependents','tenure', 'PhoneService','PaperlessBilling','MonthlyCharges','TotalCharges']]

vif_data = pd.DataFrame() vif_data["feature"] = X.columns

calculating VIF for each feature

vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(len(X.columns))]

print(vif_data)

In [55]:
```
def encode_data(dataframe):
    if dataframe.dtype == "object":
        dataframe = LabelEncoder().fit_transform(dataframe)
    return dataframe
```

```
data = data.apply(lambda x: encode_data(x))
data.head()
```

Out[55]:

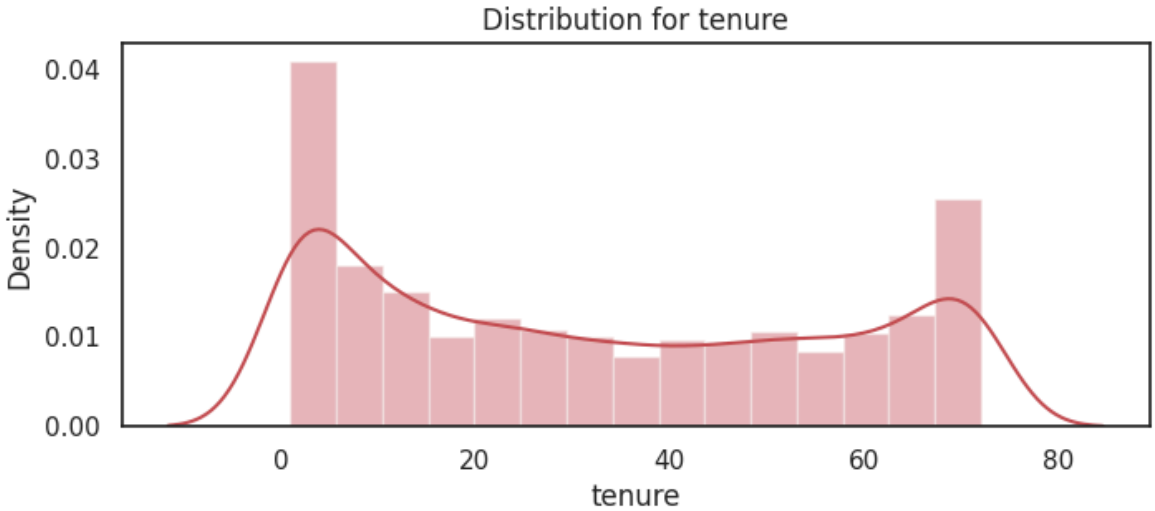
	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	In
0	0	0	1	0	1	0	1	
1	1	0	0	0	34	1	0	
2	1	0	0	0	2	1	0	
3	1	0	0	0	45	0	1	
4	0	0	0	0	2	1	0	

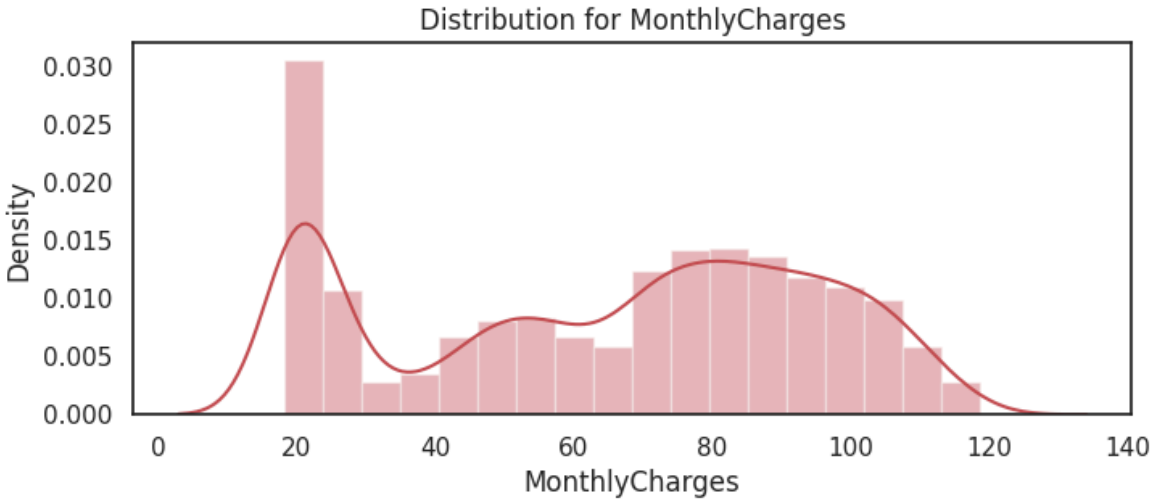
In [56]:
```
X = data.drop(columns = "Churn")
y = data["Churn"].values
```

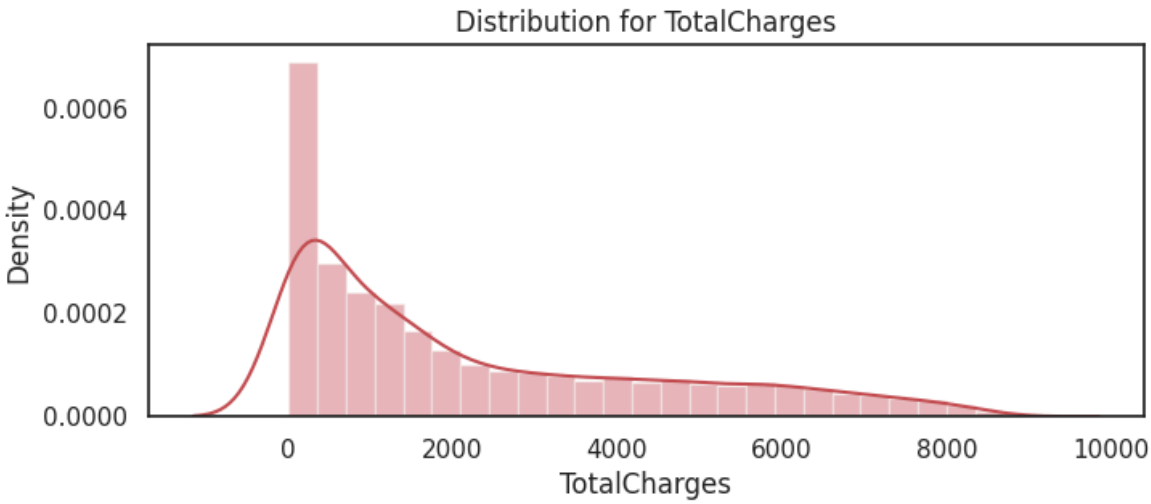
In [57]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, rando
```

In [58]:
```
def distplot(feature, frame, color='r'):
    plt.figure(figsize=(8,3))
    plt.title("Distribution for {}".format(feature))
    ax = sns.distplot(frame[feature], color= color)
```

In [59]:
```
col =  ["tenure", 'MonthlyCharges', 'TotalCharges']
for features in col :distplot(features, data)
```

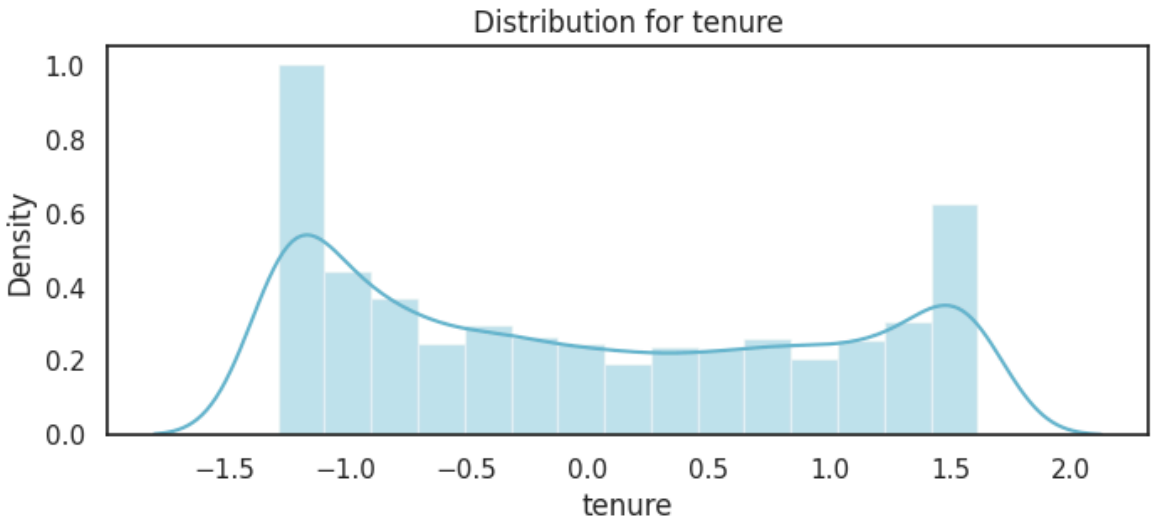


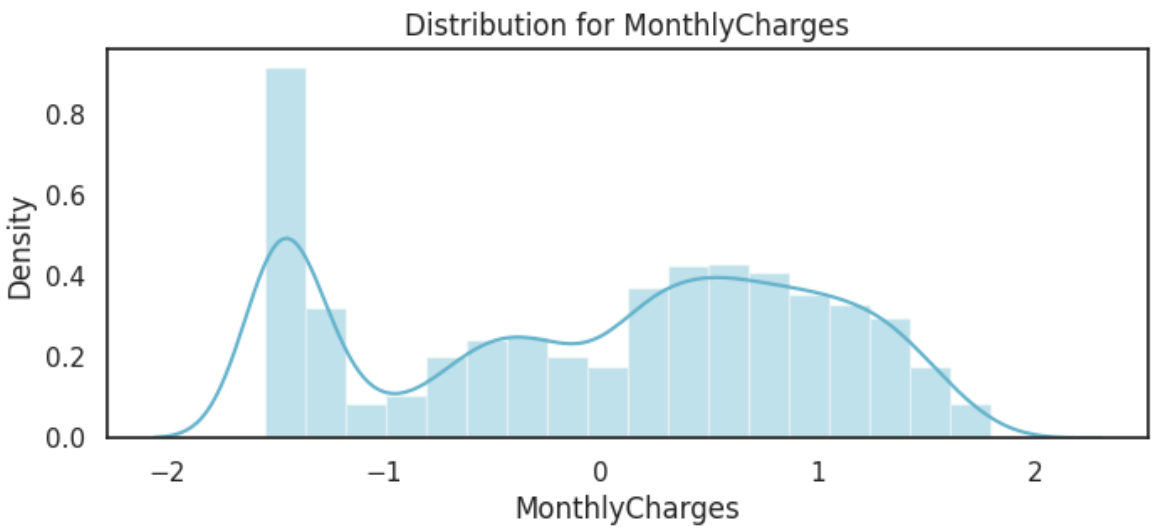


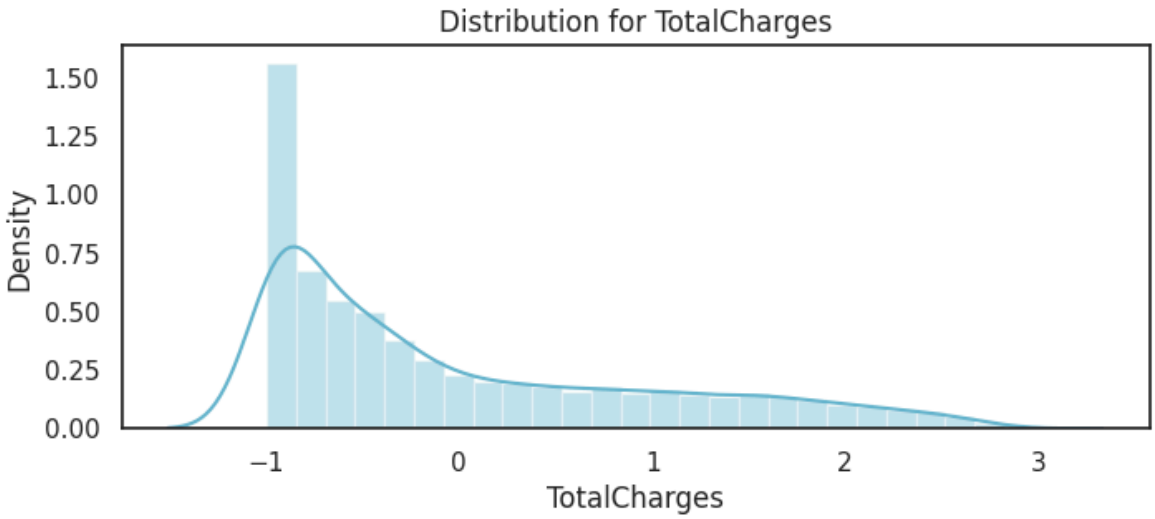


The features need standard scaling as all of them are distributed over different range values

In [60]:
```
data_std = pd.DataFrame(StandardScaler().fit_transform(data[col]).astype('float6
for feat in col: distplot(feat, data_std, color='c')
```







In [61]:
```
data.columns
```

Out[61]:
```
Index(['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',
       'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
       'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
       'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod',
       'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

In [63]:
```
for i in data.columns:
    print(i, ": ", data[i].unique())
```

```
gender :  [0 1]
SeniorCitizen :  [0 1]
Partner :  [1 0]
Dependents :  [0 1]
tenure :  [ 1 34  2 45  8 22 10 28 62 13 16 58 49 25 69 52 71 21 12 30 47 72 17 2
7
  5 46 11 70 63 43 15 60 18 66  9  3 31 50 64 56  7 42 35 48 29 65 38 68
 32 55 37 36 41  6  4 33 67 23 57 61 14 20 53 40 59 24 44 19 54 51 26 39]
PhoneService :  [0 1]
MultipleLines :  [1 0 2]
InternetService :  [0 1 2]
OnlineSecurity :  [0 2 1]
OnlineBackup :  [2 0 1]
DeviceProtection :  [0 2 1]
TechSupport :  [0 2 1]
StreamingTV :  [0 2 1]
StreamingMovies :  [0 2 1]
Contract :  [0 1 2]
PaperlessBilling :  [1 0]
PaymentMethod :  [2 3 0 1]
MonthlyCharges :  [29.85 56.95 53.85 ... 63.1  44.2  78.7 ]
TotalCharges :  [  29.85 1889.5   108.15 ...  346.45  306.6  6844.5 ]
Churn :  [0 1]
```

In [64]:
```
# Divide the columns into 3 categories, one ofor standardisation, one for label

cat_cols_ohe =['PaymentMethod', 'Contract', 'InternetService'] # those that neea
cat_cols_le = list(set(X_train.columns)- set(col) - set(cat_cols_ohe)) #those th

print(cat_cols_le)
```

```
['PhoneService', 'SeniorCitizen', 'Partner', 'Dependents', 'OnlineBackup', 'Onlin
eSecurity', 'PaperlessBilling', 'StreamingMovies', 'gender', 'DeviceProtection',
'StreamingTV', 'MultipleLines', 'TechSupport']
```

In [65]:
```
scaler = StandardScaler()
X_train[col] = StandardScaler().fit_transform(X_train[col])
X_test[col] = StandardScaler().fit_transform(X_test[col])
```

In [66]:
```
models = []

models.append(('Logistic Regression', LogisticRegression(solver='liblinear', ran
models.append(('SVC', SVC(kernel = 'linear', random_state = 0)))
models.append(('Kernel SVM', SVC(kernel = 'rbf', random_state = 0)))
models.append(('KNN', KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski'
models.append(('Gaussian NB', GaussianNB()))
models.append(('Decision Tree Classifier', DecisionTreeClassifier(criterion = 'e
models.append(('Random Forest', RandomForestClassifier(n_estimators=100, criteri
models.append(("Adaboost", AdaBoostClassifier()))
models.append(("Gradient boost classifier", GradientBoostingClassifier()))
models.append(("Voting Classifier",  VotingClassifier(estimators=[('gbc', Gradie
```

Evaluating the model Results

In [68]:
```
acc_results =[]
auc_results =[]
names = []
```

```
result_col = ["Algorithm", "ROC AUC Mean", "ROC AUC STD", "Accuracy Mean", "Accu
model_results = pd.DataFrame(columns = result_col)

i=0
# K- fold cross validation

for name, model in models:
    names.append(name)
    kfold = model_selection.KFold(n_splits=10, shuffle=True, random_state=0)

    cv_acc_results = model_selection.cross_val_score(model, X_train, y_train,
                     cv = kfold, scoring="accuracy")
    cv_auc_results = model_selection.cross_val_score(model, X_train, y_train,
                     cv = kfold, scoring="roc_auc")
    acc_results.append(cv_acc_results)
    auc_results.append(cv_auc_results)

    model_results.loc[i] = [name,
                            round(cv_auc_results.mean()*100,2),
                            round(cv_auc_results.std()*100,2),
                            round(cv_acc_results.mean()*100,2),
                            round(cv_acc_results.std()*100,2)]
    i+=1

model_results.sort_values(by = ['ROC AUC Mean'], ascending=False)
```

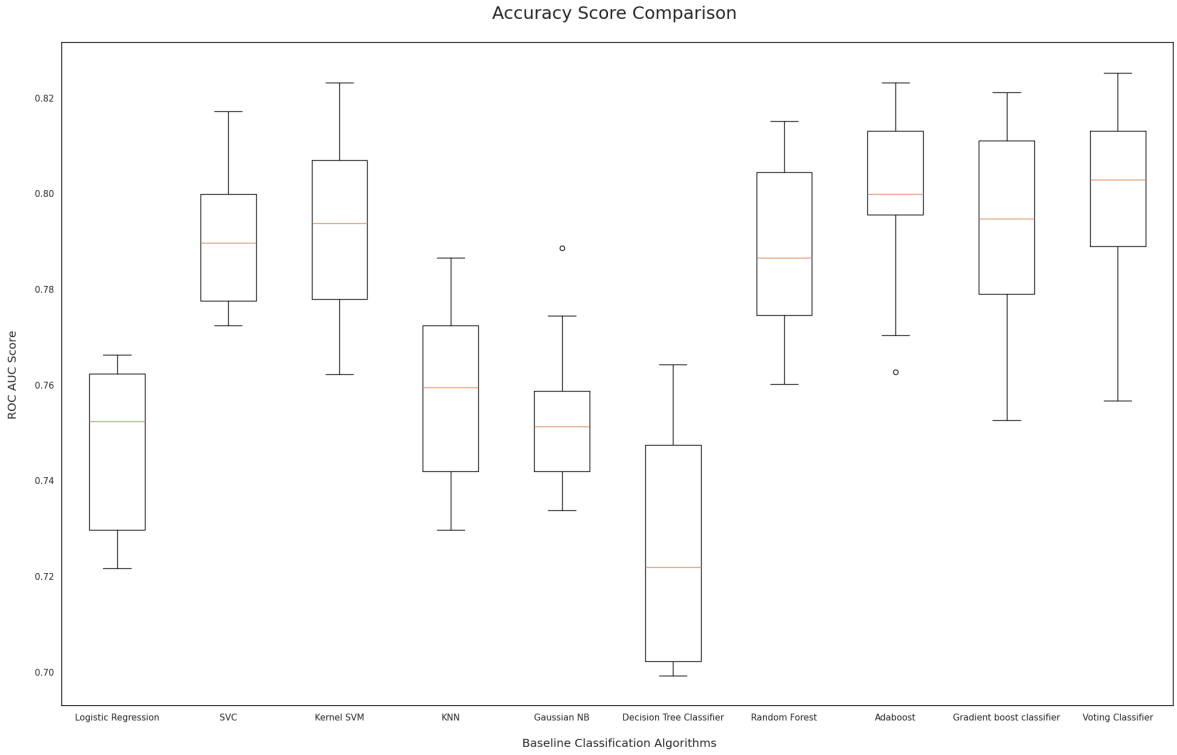
Out[68]:

	Algorithm	ROC AUC Mean	ROC AUC STD	Accuracy Mean	Accuracy STD
9	Voting Classifier	84.82	1.35	79.95	2.08
8	Gradient boost classifier	84.63	1.44	79.36	2.05
7	Adaboost	84.39	1.41	79.93	1.89
0	Logistic Regression	84.30	1.27	74.64	1.66
1	SVC	82.94	1.32	79.07	1.44
6	Random Forest	82.92	2.02	78.83	1.94
4	Gaussian NB	82.19	2.20	75.38	1.60
2	Kernel SVM	79.68	1.66	79.34	1.86
3	KNN	77.21	2.33	75.82	1.76
5	Decision Tree Classifier	65.42	2.48	72.57	2.43

In [69]:

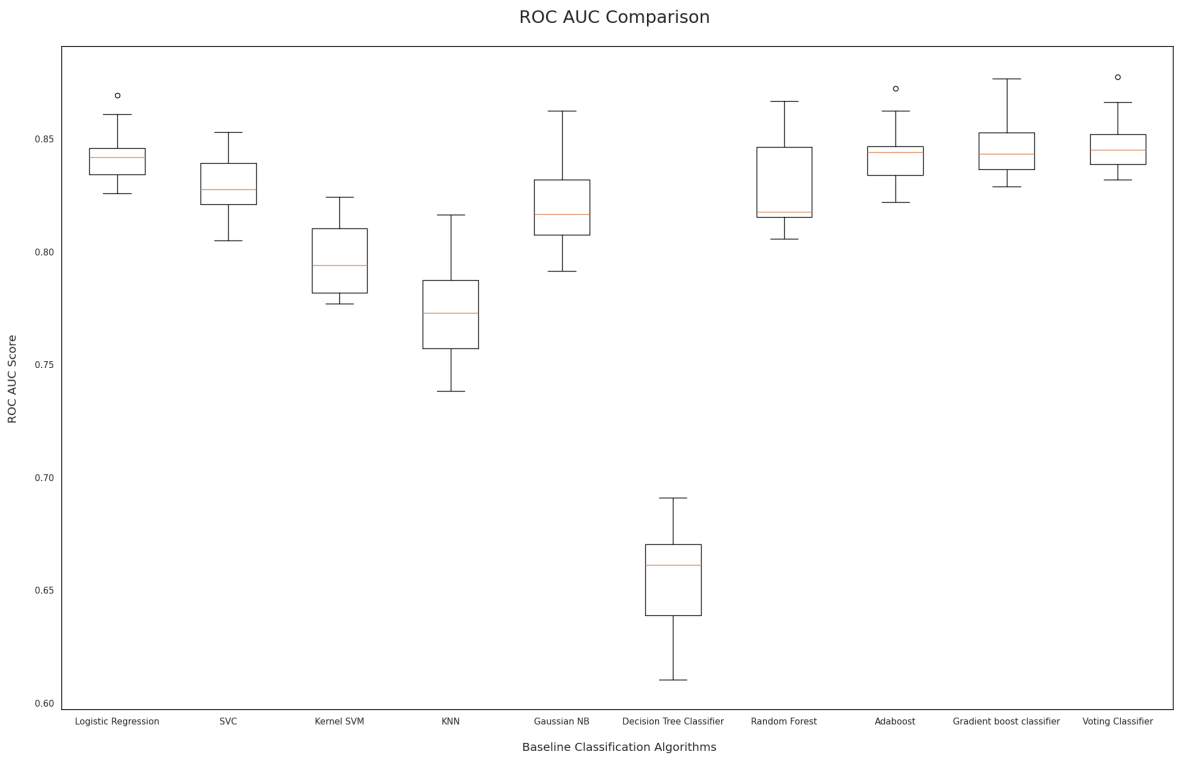
```
fig = plt.figure(figsize=(25,15))
ax = fig.add_subplot(111)
plt.boxplot(acc_results)
ax.set_xticklabels(names)

plt.ylabel('ROC AUC Score\n',
horizontalalignment="center",fontstyle = "normal",
fontsize = "large", fontfamily = "sans-serif")

plt.xlabel('\n Baseline Classification Algorithms\n',
```

```
horizontalalignment="center",fontstyle = "normal",
fontsize = "large", fontfamily = "sans-serif")

plt.title('Accuracy Score Comparison \n',
horizontalalignment="center", fontstyle = "normal",
fontsize = "22", fontfamily = "sans-serif")

plt.xticks(rotation=0, horizontalalignment="center")
plt.yticks(rotation=0, horizontalalignment="right")
plt.show()
```



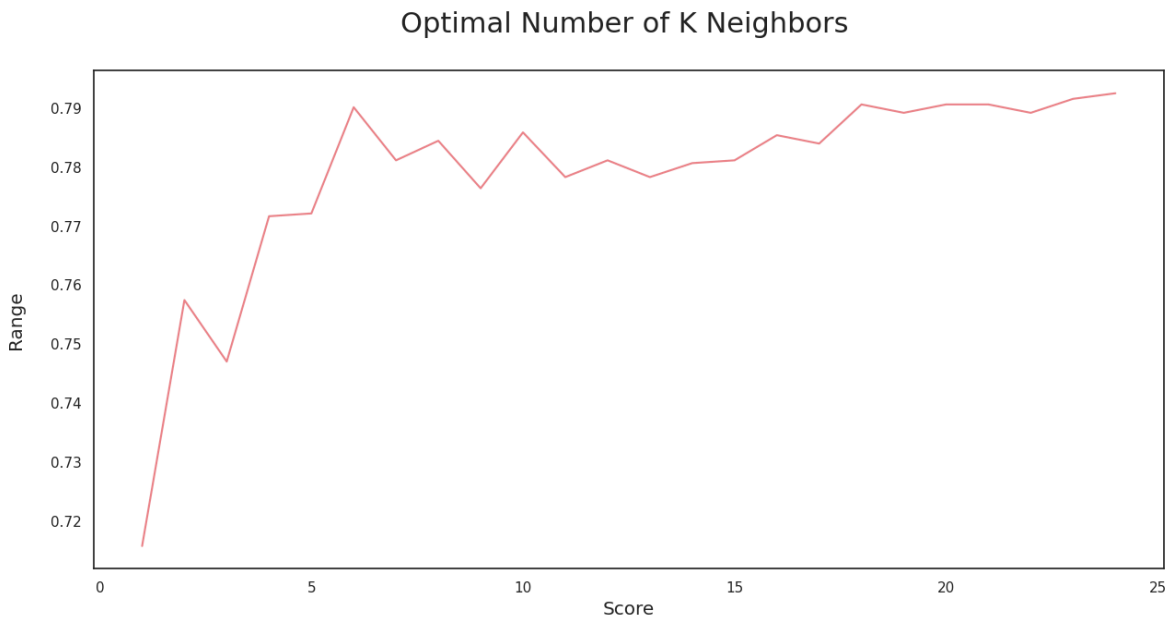
In [70]:

```
fig = plt.figure(figsize=(25,15))
ax = fig.add_subplot(111)
plt.boxplot(auc_results)
ax.set_xticklabels(names)

plt.ylabel('ROC AUC Score\n',
horizontalalignment="center",fontstyle = "normal",
fontsize = "large", fontfamily = "sans-serif")

plt.xlabel('\n Baseline Classification Algorithms\n',
horizontalalignment="center",fontstyle = "normal",
fontsize = "large", fontfamily = "sans-serif")

plt.title('ROC AUC Comparison \n',
horizontalalignment="center", fontstyle = "normal",
fontsize = "22", fontfamily = "sans-serif")

plt.xticks(rotation=0, horizontalalignment="center")
plt.yticks(rotation=0, horizontalalignment="right")
plt.show()
```



In [71]:
```
score_array = []

for each in range(1,25):
    knn_loop = KNeighborsClassifier(n_neighbors = each)
    knn_loop.fit(X_train,y_train)
    score_array.append(knn_loop.score(X_test,y_test))

score_array
```

Out[71]:
```
[0.7156398104265402,
 0.7573459715639811,
 0.7469194312796209,
 0.771563981042654,
 0.7720379146919432,
 0.7900473933649289,
 0.781042654028436,
 0.7843601895734598,
 0.776303317535545,
 0.785781990521327,
 0.7781990521327015,
 0.781042654028436,
 0.7781990521327015,
 0.7805687203791469,
 0.781042654028436,
 0.7853080568720379,
 0.7838862559241706,
 0.790521327014218,
 0.7890995260663507,
 0.790521327014218,
 0.790521327014218,
 0.7890995260663507,
 0.7914691943127962,
 0.7924170616113744]
```

KNN

In [72]:
```
fig = plt.figure(figsize=(15, 7))
plt.plot(range(1,25),score_array, color = '#ec838a')
plt.ylabel('Range\n',horizontalalignment="center",fontstyle = "normal", fontsize
plt.xlabel('Score\n',horizontalalignment="center",fontstyle = "normal", fontsize

plt.title('Optimal Number of K Neighbors \n',horizontalalignment="center", fonts
#plt.legend(loc='top right', fontsize = "medium")

plt.xticks(rotation=0, horizontalalignment="center")
plt.yticks(rotation=0, horizontalalignment="right")
plt.show()
```